

ECON1059 Seminar 2

Notes for the Seminar Series on Python Programming for Economics (ECON1059, Fall 2025)

Juncheng ZHANG, PhD Candidate in Economics, UNNC.

Research interest: Behavioral and Experimental Economics.

Email: Juncheng.ZHANG@nottingham.edu.cn

Outline

- Booleans (Multiple bools)
- if/elif/else
- input()
- while loops(for loops)

Booleans

- A **Boolean** value represents **truth** — it can be either `True` or `False` .
- Any **comparison expression** (such as `x == y` , `x > y` , `x >= y` , or `x != y`) returns a Boolean result.

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>5 == 5</code>	<code>True</code>
<code>!=</code>	Not equal to	<code>5 != 3</code>	<code>True</code>
<code>></code>	Greater than	<code>7 > 10</code>	<code>False</code>
<code><</code>	Less than	<code>3 < 8</code>	<code>True</code>
<code>>=</code>	Greater than or equal to	<code>10 >= 10</code>	<code>True</code>
<code><=</code>	Less than or equal to	<code>4 <= 2</code>	<code>False</code>

Combining Multiple Booleans

You can combine or modify Boolean values and comparison results using **logical operators**:

- **and** → Returns `True` **only if both** statements are `True` .
- **or** → Returns `True` **if at least one** statement is `True` .
- **not** → **Negates** a Boolean value (changes `True` to `False` , and vice versa).

```
In [1]: ## Example 1

print(True and False) # False because both must be True for 'and' to be True
print(True or False) # True because at least one is True for 'or' to be True
print(not True) # False 'not' reverses the value
print(not False) # True 'not' reverses the value
```

False
True
False
True

```
In [2]: ## Example 2

x = 10
y = 5

print(x == 10 and y != 10) # True (x == 10 is True, y != 10 is True)
print(x > 5 or y > 10) # True (x > 5 is True)
print(not(x < 5)) # True (x < 5 is False, but negated)
```

True
True
True

- Are the following variables equal to True or False? (Think about it first before checking it in Python)

```
In [3]: a = 7 < 3
b = -4 > 2
c = 2*(4 - 1) <= 6
d = "Jenny" == "Jennie"
e = "Marie" == "marie"
f = 888 == "888"
g = True or False
h = True and False
i = True and not False
```

```
In [4]: print("7 < 3 evaluates to:", 7 < 3)
print("-4 > 2 evaluates to:", -4 > 2)
print("2*(4-1) <= 6 evaluates to:", 2*(4 - 1) <= 6)
print('"Jenny" == "Jennie" evaluates to:', "Jenny" == "Jennie")
print('"Marie" == "marie" evaluates to:', "Marie" == "marie")
print('888 == "888" evaluates to:', 888 == "888")
print("True or False evaluates to:", True or False)
print("True and False evaluates to:", True and False)
print("True and not False evaluates to:", True and not False)
```

7 < 3 evaluates to: False
-4 > 2 evaluates to: False
2*(4-1) <= 6 evaluates to: True
"Jenny" == "Jennie" evaluates to: False
"Marie" == "marie" evaluates to: False
888 == "888" evaluates to: False
True or False evaluates to: True
True and False evaluates to: False
True and not False evaluates to: True

if/elif/else

- Used for **decision-making** in Python.
- Allows the program to **choose different actions** based on different conditions.
- Basic structure:

```
if condition1:          # Check the first condition (colon ':')
    do_something_1      # Runs if condition1 is True (indented block)
elif condition2:        # Checked only if condition1 is False
    do_something_2      # Runs if condition2 is True
else:                   # Runs only if all above conditions are False
    do_something_else
```

- `if` checks the **first condition**.
- `elif` checks **another condition** if the previous `if` was False. (You can have multiple `elif` blocks.)
- `else` runs **when all previous conditions are False**.

In [5]: `## Example 3`

```
x = 10

if x > 15:
    print("x is greater than 15")
elif x > 5:
    print("x is greater than 5 but not greater than 15")
else:
    print("x is 5 or less")
```

x is greater than 5 but not greater than 15

- Suppose a variable called `current_day` stores the current day as a text, e.g., "Monday", "Tuesday", ..., "Sunday".
 - (a) Write an if-statement that checks if the current day is Saturday. If so, a happy message should be printed to the screen. If not, a different message should be printed to the screen.
 - (b) Using the variable current day, create a bool that describes whether it's weekend.
 - (c) Create a bool that describes whether it's a weekday. Please do so in two different ways: (i) using the variable `current_day`, (ii) your answer from (b).
 - (d) Write an algorithm (which refers to any kind of code) that displays a happy message if it's weekend and another message if it's a weekday
 - (e) Write a function that checks if some day of the week (the input), falls on the weekend? Use this function to improve your answer in (d)

In [6]: `# Answer to (a). Saturday?`

```
current_day = "Wednesday"

if current_day == "Saturday":
    print("Saturdays are for play!")
else:
    print("Today is not Saturday")
```

Today is not Saturday

In [7]: `# Answer to (b). Weekend?`

```
current_day = "Sunday"
```

```
is_weekend = (current_day == "Saturday") or (current_day == "Sunday")

# Test:
print(is_weekend)
```

True

```
In [8]: # Answer to (c). Weekday?
# "The simple approach"
is_weekday = not is_weekend

# Test:
print(is_weekday)
```

False

```
In [9]: # Answer to (c). Weekday?
# "The long approach"
current_day = "Thursday"

if current_day == "Monday":
    is_weekday = True
elif current_day == "Tuesday":
    is_weekday = True
elif current_day == "Wednesday":
    is_weekday = True
elif current_day == "Thursday":
    is_weekday = True
elif current_day == "Friday":
    is_weekday = True
else:
    is_weekday = False

# Test:
print(is_weekday)
```

True

```
In [10]: # Answer to (d). Happy message if it's weekend.
current_day = "Thursday"

if (current_day == "Saturday") or (current_day == "Sunday"):
    print("It's weekend :)")
else:
    print("It's not weekend yet.")
```

It's not weekend yet.

```
In [11]: # Answer to (e). Define function to check.
def is_weekend(day):
    return (day == "Saturday") or (day == "Sunday")

current_day = "Friday"

if is_weekend(current_day):
    print("It's weekend :)")
else:
    print("It's not weekend yet.")
```

It's not weekend yet.

```
In [12]: # Answer to (e). Use the function.
def is_weekend(day):
    return (day == "Saturday") or (day == "Sunday")

current_day = input("Please enter the current day of the week: ")
if is_weekend(current_day):
    print("It's a weekend!")
else:
    print("It's a weekday.")
```

It's a weekday.

Comments on Answer to part (e)

- Notice how flexible functions can be and where they can be used. When the true/false check is a bit more complicated than our current case, this becomes very powerful!
- Indeed, reading the if-conditions suddenly becomes very easy (assuming the function has a good name that describes exactly what it does)
- When you create functions, they should usually be placed at the top. (Whether to put the "current_day" variable before or after - it really only exist to test the code - is not quite obvious)
- Write a function that takes two arguments. It should subtract the second argument from the first if both arguments are positive and returns the solution. It returns 0 if one or both arguments are negative.

```
In [13]: def subtract_two_positive_numbers(x, y):
        if x >= 0 and y >= 0:
            return x - y
        else:
            return 0
```

```
In [14]: # Let's test it
print("4-1 yields", subtract_two_positive_numbers(4, 1))
print("4-(-10) yields", subtract_two_positive_numbers(4, -10))
```

4-1 yields 3
4-(-10) yields 0

```
In [15]: def subtract_two_positive_numbers_v2(x, y):
        result = 0
        if x >= 0 and y >= 0:
            result = x - y
        return result
```

input()

- The `input()` function is used to **get input from the user**.
- It **always returns a string**, even if the user types a number.
- Therefore, you often need to **convert** the input to another type (e.g., `int()` or `float()`).
- `string.lower()` : Converts all letters to lowercase
- `string.upper()` : Converts all letters to uppercase

```
In [16]: ## Example 4
name = input("Enter your name: ")
print("Hello, " + name)

age_1 = input("Enter your age: ")
print(type(age_1))      # <class 'str'>

age_2 = int(age_1)      # Convert to integer
print(type(age_2))      # <class 'int'>

print("You will be", age_2 + 1, "next year!")
```

```
Hello, Juncheng
<class 'str'>
<class 'int'>
You will be 19 next year!
```

```
In [17]: ## Example 5

text = "hello world"

print(text.upper())     # Converts all letters to uppercase → "HELLO WORLD"
print(text.lower())     # Converts all letters to lowercase → "hello world"
```

```
HELLO WORLD
hello world
```

- Write a function that uses input() and a if/else comparison.

```
In [18]: def python_mood_check():
name = input("Hi there! What's your name?")

mood = input(f"Nice to meet you, {name}! How are you finding our Python seminar so far? (fun / hard / boring) ")

if mood.lower() == "fun":
    print("Awesome! Glad you're enjoying it – keep coding and exploring!")
elif mood.lower() == "hard":
    print("Don't worry – every great coder once felt the same. You're learning fast!")
elif mood.lower() == "boring":
    print("Uh-oh... Time to add some interactive coding challenges then!")
else:
    print("Interesting answer! Python will surprise you as you keep learning!")

print("Thanks for sharing, and happy coding!")

python_mood_check()
```

```
Uh-oh... Time to add some interactive coding challenges then!
Thanks for sharing, and happy coding!
```

for Loop

- Used to **iterate over a sequence** (like a list, tuple, string, or range).
- The number of iterations is usually **known**.

```
for variable in sequence:
    # code block to execute
```

```
In [19]: ## Example 6
for i in range(5):
    print(i)    # Prints 0, 1, 2, 3, 4
```

```
0
1
2
3
4
```

range() Function

The **range()** function in Python is used to **generate a sequence of numbers** — often for use in loops, especially **for** loops.

`range(start, stop, step)`

Parameter	Description	Default
start	First number in the sequence (included)	0
stop	Number where the sequence ends (not included)	—
step	Difference between each number (increment)	1

```
In [20]: ## Example 7
print(list(range(5)))           # [0, 1, 2, 3, 4] list() makes a list – a collection of items in order.
print(list(range(1, 6)))        # [1, 2, 3, 4, 5]
print(list(range(0, 10, 2)))     # [0, 2, 4, 6, 8]
print(list(range(10, 0, -2)))    # [10, 8, 6, 4, 2]
```

```
[0, 1, 2, 3, 4]
[1, 2, 3, 4, 5]
[0, 2, 4, 6, 8]
[10, 8, 6, 4, 2]
```

while Loop

A **while** loop repeats a block of code **as long as a given condition is True** .

It is often used when you **don't know in advance** how many times the loop should run.

```
while condition:
    # code block to repeat
    # (make sure to update something inside the loop)
```

- The condition is checked **before each iteration**.
- The code inside the loop runs **only if** the condition is **True** .
- The condition must be **updated inside the loop**, otherwise it will cause an **infinite loop**.
- **break** → Stops the loop **immediately**, even if the condition is still **True** .

- **continue** → Skips the rest of the **current iteration** and moves on to the next one.
- **else** → Runs a block of code **once after** the loop ends normally (when the condition becomes **False** or the loop finishes without **break**).

```
In [21]: ## Example 8

count = 0

while count < 5:
    print(count)
    count += 1
```

```
0
1
2
3
4
```

```
In [22]: ## Example 9

i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

```
1
2
3
```

```
In [23]: ## Example 10

i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

```
1
2
4
5
6
```

```
In [24]: ## Example 11

i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```

```
1
2
3
4
5
i is no longer less than 6
```

Aspect	for Loop	while Loop
Usage	For known number of iterations	For unknown repetitions until a condition fails
Control	Iterates over a sequence	Runs while a condition is True
Setup	Counter defined in the loop	Variable defined before the loop
Risk	Rarely infinite	Can be infinite if condition never changes

- For numbers $x = 3, 4, 5, 6, 7$, calculate $3 \cdot x + 5$ and print the results.

```
In [25]: x = 3
while x <= 7:
    print("For x=", x, ", 3*x+5=", 3*x + 5)
    x = x + 1
```

```
For x= 3 , 3*x+5= 14
For x= 4 , 3*x+5= 17
For x= 5 , 3*x+5= 20
For x= 6 , 3*x+5= 23
For x= 7 , 3*x+5= 26
```

- For numbers $x = 2, 4, 6, 8, \dots, 10$, calculate $x^2 - 1$ and print the results.

```
In [26]: x = 2
while x <= 10:
    print("For x=", x, ", x^2-1=", x**2 - 1)
    x += 2
```

```
For x= 2 , x^2-1= 3
For x= 4 , x^2-1= 15
For x= 6 , x^2-1= 35
For x= 8 , x^2-1= 63
For x= 10 , x^2-1= 99
```

- In the lecture, I wrote the following code that asked people to input a number between 1 and 10:

```
In [27]: number = input("Please enter a number from 1 to 10:")
number = int(number)
if number > 0 and number < 11:
    print("Well done, you entered a suitable number :)")
else:
    print("A little rebel you are, aren't you?")
```

Well done, you entered a suitable number :)

- I want you to write a nicer version of this, where, if the user doesn't enter a correct number, s/he is asked to do so again until s/he enters a valid number.
- Note, this question can be quite tricky. Consider writing what needs to happen in plain english before you code it.
- (Extra feature: can you track how many times it took the user to enter the number and include this in your print statements.)

```
In [28]: requires_input = True
num_failed_inputs = 0 # Extra feature
```

```

while requires_input:
    number_in = input("Please enter a number from 1 to 10:")
    number = int(number_in)

    if number > 0 and number < 11:
        requires_input = False
        print("Well done, you entered a correct number.")
        print("You did it in", 1+num_failed_inputs, "attempts.")
    else:
        print("Wrong entry. Try again please.")
        num_failed_inputs +=1

```

Well done, you entered a correct number.
 You did it in 1 attempts.

Appendix

- `for` Loop to check whether a coin is fair

(We haven't learned all the code here yet — for now, just know that `for` loops are very useful in simulations, so don't worry about the details!).

```

In [29]: import random

heads = 0
tails = 0

# Flip the coin () times
for i in range(10000):
    # random.choice() selects one element from the list uniformly at random.
    # Here, "Heads" and "Tails" each have probability 0.5.
    flip = random.choice(["Heads", "Tails"])
    if flip == "Heads":
        heads += 1
    else:
        tails += 1

# Print the counts
print("Heads:", heads)
print("Tails:", tails)

```

Heads: 4983
 Tails: 5017